

EMBEDDED AND REAL TIME SYSTEMS

Example of Real Time Operating Systems

Example of RTOS

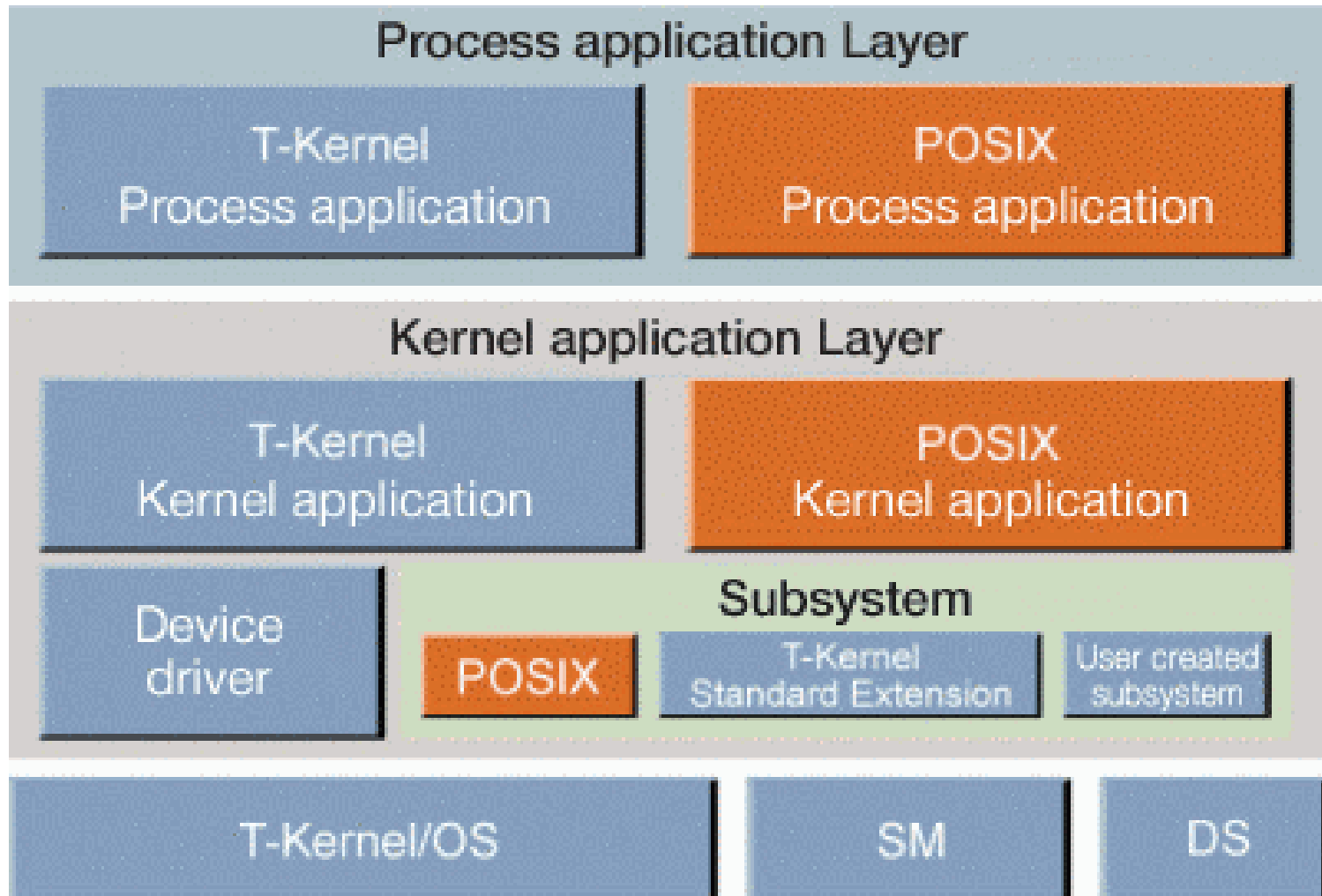
- POSIX
- Windows CE

POSIX

- POSIX - **Portable Operating System Interface**
 - POSIX.1 – Core services
 - POSIX.1b – Real-time extensions
 - POSIX.1c – Thread extensions
- The POSIX standard is written in terms of the C programming language. POSIX supports two programming environments.
- One is based on the **Traditional C language**.
- The other is based on the **Standard C language** defined by American National Standard for Information Systems.
- Since Standard C was developed by the **American National Standards Institute (ANSI)**, some people call it ANSI C

- POSIX is based on **UNIX** operating system.
- Usually UNIX was not originally designed as a real time operating system, POSIX has been extended to support real time requirements.
- **POSIX defines** a standard way for an application to interface to the operating system.
- The original POSIX standard defines interfaces to core functions such as file operations, process management, signals, and devices.

POSIX



The POSIX process model

- POSIX does **not** implement **lightweight processes**.
- Each POSIX process runs in its **own address space** and **cannot** directly access the data or code of **other processes**

Processes in POSIX

- In POSIX, a **new process** is created by making a **copy** of an **existing process**
- The **copying process** creates two different processes both running the same code
- The complication comes in ensuring that one process runs the code intended for the new process.
- While the other process continues the work of the old process.

- A process makes a copy of itself by calling **fork()** function
- It creates **a new child process which is exact copy of parent process**
- The both have the same code and the same data values with one exceptions the **return value** of **fork()**
 - **Parent Process:** returns the process ID of the child process
 - **Child process:** returns 0
- We can Test the return value of **fork()** to determine which process is the child.

- **execv()** function -> It takes the list of arguments to the process in the form of **array**.
- **perror() & exit()** function -> To perform **call functions**.
- **parent_stuff()** function -> To perform the work of **parent function**.
- **wait()** function -> It make **wait the child process**

Real time scheduling in POSIX

- POSIX supports real time scheduling in the **_POSIX_PRIORITY_SCHEDULING** resource
- The **sched_setscheduler()** function is used to determine a process's scheduling policy and other parameters

Ex: `i = sched_setscheduler (process_id, SCHED_FIFO, &sched_params)`

- It tells POSIX to use the **SCHED_FIFO** policy for this process along with some other scheduling parameters.
- ❖ **sched_getparams()** function returns the current parameter values for a process
- ❖ **sched_setparams()** function changes the parameter values

SCHED_FIFO scheduling

- POSIX supports **rate-monotonic scheduling in the SCHED_FIFO scheduling policy.**
- It is a **strict priority-based**
- scheduling scheme in which a process runs **until** it is **preempted or terminates**
- The term FIFO simply refers to the fact that, within a priority, processes run in **first-come first-served order.**

SCHED_RR

- SCHED_RR is a combination of **real-time and interactive scheduling techniques**:
- within a **priority level**, the processes are **timesliced**.
Interactive systems must ensure
- that all processes get a chance to run, so time is divided into quanta. Processes get the
- CPU in multiple of quanta

SCHED_OTHER

- The SCHED_OTHER is defined to allow **non-real-time processes to intermix with realtime processes.**
- The precise scheduling mechanism used by this policy is not defined.
- It is used to indicate that the process **does not need a real-time scheduling policy.**

POSIX

- POSIX supports
 - Semaphores
 - Shared memory mechanism
 - Message Queues

POSIX semaphores

- POSIX supports Counting semaphores in the `_POSIX_SEMAPHORES` option
- A Counting semaphore allows more than one process to access a resource at a time
- If a semaphore allows up to N resources, then it will not block until N process have simultaneously passed the semaphore

semaphores

- At that point, the **blocked process can resume** only after one of the processes has given up its semaphore.
- The simplest way to think about counting semaphores is that they **count down to 0-** when the **semaphore value is 0-**,
- the **process must wait** until **another process** gives up the semaphore and **increments the count.**

POSIX semaphores

- Names for the semaphore start with “/”
- `sem_open()` – To create a semaphore
- `sem_close ()` – To destroy a semaphore
- `sem_wait()` – To wait for Getting a semaphore
- `sem_post()` – To Releasing a semaphore

POSIX Shared Memory

- POSIX shared memory is supported under the `_POSIX_SHARED_MEMORY_OBJECTS` option.
- Shared memory functions **create blocks of memory that can be used by several processes**
 - `shm_open()` – To open a shared memory
 - `close()` – To close a shared memory
 - `mmap()` – For memory mapping
 - `objdesc = shm_open("/memobj1", O_RDWR);`

- Before using the shared memory object, we must **map it into the process memory**
- space using the **mmap() function**. POSIX assumes that **shared memory objects**
- fundamentally reside in a backing store such as a disk and are then **mapped into the address space of the process.**

POSIX pipes

- The pipe is very familiar to Unix users from its shell syntax:

```
if (pipe(pipe_ends) < 0) { /* create the pipe, check for errors */  
    perror("pipe");  
    break;
```

- A parent **process uses the pipe() function** to create a **pipe to talk to a child**.
- It must do so before the child itself is created or it will not have any way to pass **a pointer to the pipe to the child**.

- Each end of a pipe appears to the programs as a **file** **the process** at the head of the pipe **writes to one file descriptor** while the tail process reads from another file descriptor.
- The `pipe()` function returns an **array of file descriptors**,
- The first for the **write end** and the second for the **read end**.

POSIX Message Queues

- POSIX supports **message queues** under the **_POSIX_MESSAGE_PASSING** option.
- Message queues starts with “/”
- No need to create a queue before creating a process

- **mq_open()** – to create named queue
- **mq_close()** – to destroy named queue

- **mq_send()** – to transmit a message
- **mq_receive()** – to receive a message

- **mq_maxmsg()** – Maximum number of messages
- **mq_msgsize()** – Maximum size of a message

- The advantage of a **queue over a pipe** is that, because **queues have names**,
- we **do not** have to **create the pipe descriptor** before creating the other process using it, as with pipes.
- The name of a queue follows the **same rules as for semaphores and shared memory**: it starts with a “/” and contains no other “/” characters